

gorithm predicts 0 and the outcome is 1. Without loss of generality, assume that the algorithm predicts 1 and the outcome is 0. The algorithm predicted 1 because $upweight^{(t)} \geq downweight^{(t)}$ in line 9, which implies that

$$upweight^{(t)} \geq W(t)/2. \quad (33.7)$$

Each expert in U then has its weight multiplied by $1 - \gamma$, and each expert in D has its weight unchanged. Thus, we have

$$\begin{aligned} W(t+1) &= upweight^{(t)}(1 - \gamma) + downweight^{(t)} \\ &= upweight^{(t)} + downweight^{(t)} - \gamma \cdot upweight^{(t)} \\ &= W(t) - \gamma \cdot upweight^{(t)} \\ &\leq W(t) - \gamma \frac{W(t)}{2} \quad (\text{by inequality (33.7)}) \\ &= W(t)(1 - \gamma/2). \end{aligned}$$

Therefore, for every iteration t in which the algorithm makes a mistake, we have

$$W(t+1) \leq (1 - \gamma/2)W(t). \quad (33.8)$$

In an iteration where the algorithm does not make a mistake, some of the weights decrease and some remain unchanged, so that we have

$$W(t+1) \leq W(t). \quad (33.9)$$

Since there are $m^{(T')}$ mistakes made through iteration T' , and $W(1) = n$, we can repeatedly apply inequality (33.8) to iterations where the algorithm makes a mistake and inequality (33.9) to iterations where the algorithm does not make a mistake, obtaining

$$W(T') \leq n(1 - \gamma/2)^{m^{(T')}}. \quad (33.10)$$

Because the function W is the sum of the weights and all weights are positive, its value exceeds any single weight. Therefore, using equation (33.6) we have, for any expert E_i and for any iteration $T' \leq T$,

$$W(T') \geq w_i^{(T')} = (1 - \gamma)^{m_i^{(T')}}. \quad (33.11)$$

Combining inequalities (33.10) and (33.11) gives

$$(1 - \gamma)^{m_i^{(T')}} \leq n(1 - \gamma/2)^{m^{(T')}}.$$

Taking the natural logarithm of both sides yields

$$m_i^{(T')} \ln(1 - \gamma) \leq m^{(T')} \ln(1 - \gamma/2) + \ln n. \quad (33.12)$$

We now use the Taylor series expansion to derive upper and lower bounds on the logarithmic factors in inequality (33.12). The Taylor series for $\ln(1+x)$ is given in equation (3.22) on page 67. Substituting $-x$ for x , we have that for $0 < x \leq 1/2$,

$$\ln(1-x) = -x - \frac{x^2}{2} - \frac{x^3}{3} - \frac{x^4}{4} - \dots \quad (33.13)$$

Since each term on the right-hand side is negative, we can drop all terms except the first and obtain an upper bound of $\ln(1-x) \leq -x$. Since $0 < \gamma \leq 1/2$, we have

$$\ln(1-\gamma/2) \leq -\gamma/2 \quad (33.14)$$

For the lower bound, Exercise 33.2-2 asks you to show that $\ln(1-x) \geq -x - x^2$ when $0 < x \leq 1/2$, so that

$$-\gamma - \gamma^2 \leq \ln(1-\gamma) \quad (33.15)$$

Thus, we have

$$\begin{aligned} m_i^{(T')}(-\gamma - \gamma^2) &\leq m_i^{(T')} \ln(1-\gamma) && \text{(by inequality (33.15))} \\ &\leq m^{(T')} \ln(1-\gamma/2) + \ln n && \text{(by inequality (33.12))} \\ &\leq m^{(T')}(-\gamma/2) + \ln n && \text{(by inequality (33.14))} , \end{aligned}$$

so that

$$m_i^{(T')}(-\gamma - \gamma^2) \leq m^{(T')}(-\gamma/2) + \ln n \quad (33.16)$$

Subtracting $\ln n$ from both sides of inequality (33.16) and then multiplying both sides by $-2/\gamma$ yields $m^{(T')} \leq 2(1+\gamma)m_i^{(T')} + (2\ln n)/\gamma$, thus proving the theorem. ■

Theorem 33.4 applies to any expert and any event $T' \leq T$. In particular, we can compare against the best expert after all events have occurred, producing the following corollary.

Corollary 33.5

At the end of procedure WEIGHTED-MAJORITY, we have

$$m^{(T)} \leq 2(1+\gamma)m^* + \frac{2\ln n}{\gamma} \quad (33.17)$$

■

Let's explore this bound. Assuming that $\sqrt{\ln n/m^*} \leq 1/2$, we can choose $\gamma = \sqrt{\ln n/m^*}$ and plug into inequality (33.17) to obtain

$$\begin{aligned}
m^{(T)} &\leq 2 \left(1 + \sqrt{\frac{\ln n}{m^*}} \right) m^* + \frac{2 \ln n}{\sqrt{\ln n / m^*}} \\
&= 2m^* + 2\sqrt{m^* \ln n} + 2\sqrt{m^* \ln n} \\
&= 2m^* + 4\sqrt{m^* \ln n} ,
\end{aligned}$$

and so the number of errors is at most twice the number of errors made by the best expert plus a term that is often slower growing than m^* . Exercise 33.2-4 shows that you can decrease the bound on the number of errors by a factor of 2 by using randomization, which leads to much stronger bounds. In particular, the upper bound on regret $(m - m^*)$ is reduced from $(1 + 2\gamma)m^* + (2 \ln n)/\gamma$ to an expected value of $\epsilon m^* + (\ln n)/\epsilon$, where both γ and ϵ are at most $1/2$. Numerically, we can see that if $\gamma = 1/2$, WEIGHTED-MAJORITY makes at most 3 times the number of errors as the best expert, plus $4 \ln n$ errors. As another example, suppose that $T = 1000$ predictions are being made by $n = 20$ experts, and the best expert is correct 95% of the time, making 50 errors. Then WEIGHTED-MAJORITY makes at most $100(1 + \gamma) + 2 \ln 20 / \gamma$ errors. By choosing $\gamma = 1/4$, WEIGHTED-MAJORITY makes at most 149 errors, or a success rate of at least 85%.

Multiplicative weights methods typically refer to a broader class of algorithms that includes WEIGHTED-MAJORITY. The outcomes and predictions need not be only 0 or 1, but can be real numbers, and there can be a loss associated with a particular outcome and prediction. The weights can be updated by a multiplicative factor that depends on the loss, and the algorithm can, given a set of weights, treat them as a distribution on experts and use them to choose an expert to follow in each event. Even in these more general settings, bounds similar to Theorem 33.4 hold.

Exercises

33.2-1

The proof of Lemma 33.3 assumes that some expert never makes a mistake. It is possible to generalize the algorithm and analysis to remove this assumption. The new algorithm begins in the same way. The set S might become empty at some point, however. If that ever happens, reset S to contain all the experts and continue the algorithm. Show that the number of mistakes that this algorithm makes is at most $m^* \lceil \lg n \rceil$.

33.2-2

Show that $\ln(1 - x) \geq -x - x^2$ when $0 < x \leq 1/2$. (*Hint:* Start with equation (33.13), group all the terms after the first three, and use equation (A.7) on page 1142.)

33.2-3

Consider a randomized variant of the algorithm given in the proof of Lemma 33.3, in which some expert never makes a mistake. At each step, choose an expert E_i uniformly at random from the set S and then make the same prediction as E_i . Show that the expected number of mistakes made by this algorithm is $\lceil \lg n \rceil$.

33.2-4

Consider a randomized version of WEIGHTED-MAJORITY. The algorithm is the same, except for the prediction step, which interprets the weights as a probability distribution over the experts and chooses an expert E_i according to that distribution. It then chooses its prediction to be the same as the prediction made by expert E_i . Show that, for any $0 < \epsilon < 1/2$, the expected number of mistakes made by this algorithm is at most $(1 + \epsilon)m^* + (\ln n)/\epsilon$.

33.3 Gradient descent

Suppose that you have a set $\{p_1, p_2, \dots, p_n\}$ of points and you want to find the line that best fits these points. For any line ℓ , there is a distance d_i between each point p_i and the line. You want to find the line that minimizes some function $f(d_1, \dots, d_n)$. There are many possible choices for the definition of distance and for the function f . For example, the distance can be the projection distance to the line and the function can be the sum of the squares of the distances. This type of problem is common in data science and machine learning—the line is the hypothesis that best describes the data—where the particular definition of best is determined by the definition of distance and the objective f . If the definition of distance and the function f are linear, then we have a linear-programming problem, as discussed in Chapter 29. Although the linear-programming framework captures several important problems, many other problems, including various machine-learning problems, have objectives and constraints that are not necessarily linear. We need frameworks and algorithms to solve such problems.

In this section, we consider the problem of optimizing a continuous function and discuss one of the most popular methods to do so: gradient descent. Gradient descent is a general method for finding a local minimum of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, where informally, a local minimum of a function f is a point $\mathbf{x}$ for which $f(\mathbf{x}) \leq f(\mathbf{x}')$ for all $\mathbf{x}'$ that are “near” $\mathbf{x}$. When the function is convex, it can find a point near the *global minimizer* of f : an n -vector argument $\mathbf{x} = (x_1, x_2, \dots, x_n)$ such that $f(\mathbf{x})$ is minimum. For the intuitive idea behind gradient descent, imagine being in a landscape of hills and valleys, and wanting to get to a low point as quickly as possible. You survey the terrain and choose to

move in the direction that takes you downhill the fastest from your current position. You move in that direction, but only for a short while, because as you proceed, the terrain changes and you might need to choose a different direction. So you stop, reevaluate the possible directions and move another short distance in the steepest downhill direction, which might differ from the direction of your previous movement. You continue this process until you reach a point from which all directions lead up. Such a point is a local minimum.

In order to make this informal procedure more formal, we need to define the gradient of a function, which in the analogy above is a measure of the steepness of the various directions. Given a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, its **gradient** ∇f is a function $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ comprising n partial derivatives: $(\nabla f)(\mathbf{x}) = (\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n})$. Analogous to the derivative of a function of a single variable, the gradient can be viewed as a direction in which the function value locally increases the fastest, and the rate of that increase. This view is informal; in order to make it formal we would have to define what local means and place certain conditions, such as continuity or existence of derivatives, on the function. Nevertheless, this view motivates the key step of gradient descent—move in the direction opposite to the gradient, by a distance influenced by the magnitude of the gradient.

The general procedure of gradient descent proceeds in steps. You start at some initial point $\mathbf{x}^{(0)}$, which is an n -vector. At each step t , you compute the value of the gradient of f at point $\mathbf{x}^{(t)}$, that is, $(\nabla f)(\mathbf{x}^{(t)})$, which is also an n -vector. You then move in the direction opposite to the gradient in each dimension at $\mathbf{x}^{(t)}$ to arrive at the next point $\mathbf{x}^{(t+1)}$, which again is an n -vector. Because you moved in a monotonically decreasing direction in each dimension, you should have that $f(\mathbf{x}^{(t+1)}) \leq f(\mathbf{x}^{(t)})$. Several details are needed to turn this idea into an actual algorithm. The two main details are that you need an initial point and that you need to decide how far to move in the direction of the negative gradient. You also need to understand when to stop and what you can conclude about the quality of the solution found. We will explore these issues further in this section, for both constrained minimization, where there are additional constraints on the points, and unconstrained minimization, where there are none.

Unconstrained gradient descent

In order to gain intuition, let's consider unconstrained gradient descent in just one dimension, that is, when f is a function of a scalar x , so that $f : \mathbb{R} \rightarrow \mathbb{R}$. In this case, the gradient ∇f of f is just $f'(x)$, the derivative of f with respect to x . Consider the function f shown in blue in Figure 33.3, with minimizer x^* and starting point $x^{(0)}$. The gradient (derivative) $f'(x^{(0)})$, shown in orange, has a negative slope, so that a small step from $x^{(0)}$ in the direction of increasing x results in a point x' for which $f(x') < f(x^{(0)})$. Too large a step, however, results in a

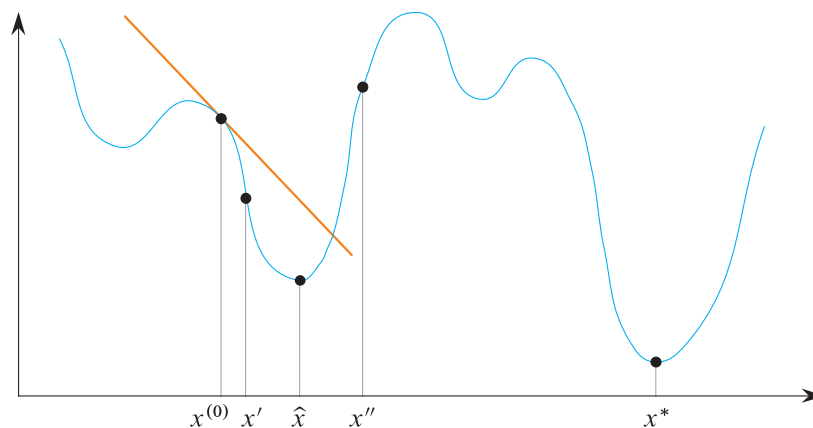


Figure 33.3 A function $f : \mathbb{R} \rightarrow \mathbb{R}$, shown in blue. Its gradient at point $x^{(0)}$, in orange, has a negative slope, and so a small increase in x from $x^{(0)}$ to x' results in $f(x') < f(x^{(0)})$. Small increases in x from $x^{(0)}$ head toward $\hat{x}$, which gives a local minimum. Too large an increase in x can end up at x'' , where $f(x'') > f(x^{(0)})$. Small steps starting from $x^{(0)}$ and going only in the direction of decreasing values of f cannot end up at the global minimizer x^* .

point x'' for which $f(x'') > f(x^{(0)})$, so this is a bad idea. Restricting ourselves to small steps, where each one has $f(x') < f(x)$, eventually results in getting close to point $\hat{x}$, which gives a local minimum. By taking only small downhill steps, however, gradient descent has no chance to get to the global minimizer x^* , given the starting point $x^{(0)}$.

We draw two observations from this simple example. First, gradient descent converges toward a local minimum, and not necessarily a global minimum. Second, the speed at which it converges and how it behaves are related to properties of the function, to the initial point, and to the step size of the algorithm.

The procedure GRADIENT-DESCENT on the facing page takes as input a function f , an initial point $\mathbf{x}^{(0)} \in \mathbb{R}^n$, a fixed step-size multiplier $\gamma > 0$, and a number $T > 0$ of steps to take. Each iteration of the **for** loop of lines 2–4 performs a step by computing the n -dimensional gradient at point $\mathbf{x}^{(t)}$ and then moving distance γ in the opposite direction in the n -dimensional space. The complexity of computing the gradient depends on the function f and can sometimes be expensive. Line 3 sums the points visited. After the loop terminates, line 6 returns $\mathbf{x}\text{-avg}$, the average of all the points visited except for the last one, $\mathbf{x}^{(T)}$. It might seem more natural to return $\mathbf{x}^{(T)}$, and in fact, in many circumstances, you might prefer to have the function return $\mathbf{x}^{(T)}$. For the version we will analyze, however, we use $\mathbf{x}\text{-avg}$.

```

GRADIENT-DESCENT( $f, \mathbf{x}^{(0)}, \gamma, T$ )
1  sum = 0 //  $n$ -dimensional vector, initially all 0
2  for  $t = 0$  to  $T - 1$ 
3      sum = sum +  $\mathbf{x}^{(t)}$  // add each of  $n$  dimensions into sum
4       $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \gamma \cdot (\nabla f)(\mathbf{x}^{(t)})$  //  $(\nabla f)(\mathbf{x}^{(t)})$ ,  $\mathbf{x}^{(t+1)}$  are  $n$ -dimensional
5  x-avg = sum /  $T$  // divide each of  $n$  dimensions by  $T$ 
6  return x-avg

```

Figure 33.4 depicts how gradient descent ideally runs on a convex 1-dimensional function.¹ We'll define convexity more formally below, but the figure shows that each iteration moves in the direction opposite to the gradient, with the distance moved being proportional to the magnitude of the gradient. As the iterations proceed, the magnitude of the gradient decreases, and thus the distance moved along the horizontal axis decreases. After each iteration, the distance to the optimal point $\mathbf{x}^*$ decreases. This ideal behavior is not guaranteed to occur in general, but the analysis in the remainder of this section formalizes when this behavior occurs and quantifies the number of iterations needed. Gradient descent does not always work, however. We have already seen that if the function is not convex, gradient descent can converge to a local, rather than global, minimum. We have also seen that if the step size is too large, GRADIENT-DESCENT can overshoot the minimum and wind up farther away. (It is also possible to overshoot the minimum and wind up closer to the optimum.)

Analysis of unconstrained gradient descent for convex functions

Our analysis of gradient descent focuses on convex functions. Inequality (C.29) on page 1194 defines a convex function of one variable, as shown in Figure 33.5. We can extend that definition to a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and say that f is *convex* if for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and for all $0 \leq \lambda \leq 1$, we have

$$f(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y}). \quad (33.18)$$

(Inequalities (33.18) and (C.29) are the same, except for the dimensions of $\mathbf{x}$ and $\mathbf{y}$.) We also assume that our convex functions are closed² and differentiable.

¹ Although the curve in Figure 33.4 looks concave, according to the definition of convexity that we'll see below, the function f in the figure is convex.

² A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is closed if, for each $\alpha \in \mathbb{R}$, the set $\{\mathbf{x} \in \text{dom}(f) : f(\mathbf{x}) \leq \alpha\}$ is closed, where $\text{dom}(f)$ is the domain of f .

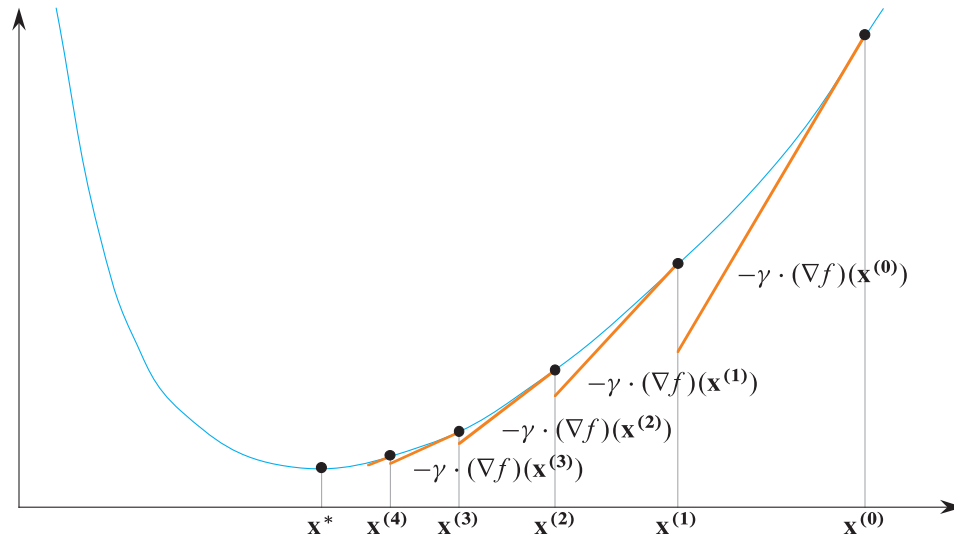


Figure 33.4 An example of running gradient descent on a convex function $f : \mathbb{R} \rightarrow \mathbb{R}$, shown in blue. Beginning at point $\mathbf{x}^{(0)}$, each iteration moves in the direction opposite to the gradient, and the distance moved is proportional to the magnitude of the gradient. Orange lines represent the negative of the gradient at each point, scaled by the step size γ . As the iterations proceed, the magnitude of the gradient decreases, and the distance moved decreases correspondingly. After each iteration, the distance to the optimal point $\mathbf{x}^*$ decreases.

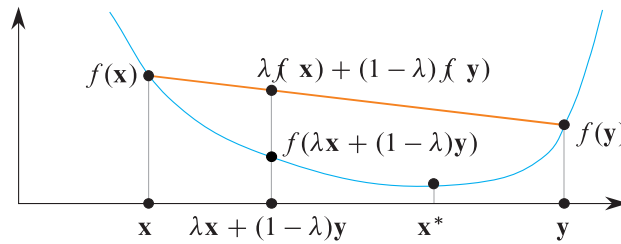


Figure 33.5 A convex function $f : \mathbb{R} \rightarrow \mathbb{R}$, shown in blue, with local and global minimizer $\mathbf{x}^*$. Because f is convex, $f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y})$ for any two values $\mathbf{x}$ and $\mathbf{y}$ and all $0 \leq \lambda \leq 1$, shown for a particular value of λ . Here, the orange line segment represents all values $\lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y})$ for $0 \leq \lambda \leq 1$, and it is above the blue line.

A convex function has the property that any local minimum is also a global minimum. To verify this property, consider inequality (33.18), and suppose for the purpose of contradiction that $\mathbf{x}$ is a local minimum but not a global minimum and $\mathbf{y} \neq \mathbf{x}$ is a global minimum, so $f(\mathbf{y}) < f(\mathbf{x})$. Then we have

$$\begin{aligned} f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) &\leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}) \quad (\text{by inequality (33.18)}) \\ &< \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{x}) \\ &= f(\mathbf{x}) . \end{aligned}$$

Thus, letting λ approach 1, we see that there is another point near $\mathbf{x}$, say $\mathbf{x}'$, such that $f(\mathbf{x}') < f(\mathbf{x})$, so $\mathbf{x}$ is not a local minimum.

Convex functions have several useful properties. The first property, whose proof we leave as Exercise 33.3-1, says that a convex function always lies above its tangent hyperplane. In the context of gradient descent, angle brackets denote the notation for inner product defined on page 1219 rather than denoting a sequence.

Lemma 33.6

For any convex differentiable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, we have $f(\mathbf{x}) \leq f(\mathbf{y}) + \langle (\nabla f)(\mathbf{x}), \mathbf{x} - \mathbf{y} \rangle$. ■

The second property, which Exercise 33.3-2 asks you to prove, is a repeated application of the definition of convexity in inequality (33.18).

Lemma 33.7

For any convex function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, for any integer $T \geq 1$, and for all $\mathbf{x}^{(0)}, \dots, \mathbf{x}^{(T-1)} \in \mathbb{R}^n$, we have

$$f\left(\frac{\mathbf{x}^{(0)} + \dots + \mathbf{x}^{(T-1)}}{T}\right) \leq \frac{f(\mathbf{x}^{(0)}) + \dots + f(\mathbf{x}^{(T-1)})}{T}. \quad (33.19)$$

■

The left-hand side of inequality (33.19) is the value of f at the vector $\mathbf{x}\text{-avg}$ that GRADIENT-DESCENT returns.

We now proceed to analyze GRADIENT-DESCENT. It might not return the exact global minimizer $\mathbf{x}^*$. We use an error bound ϵ , and we want to choose T so that $f(\mathbf{x}\text{-avg}) - f(\mathbf{x}^*) \leq \epsilon$ at termination. The value of ϵ depends on the number T of iterations and two additional values. First, since you expect it to be better to start close to the global minimizer, ϵ is a function of

$$R = \|\mathbf{x}^{(0)} - \mathbf{x}^*\|, \quad (33.20)$$

the euclidean norm (or distance, defined on page 1219) of the difference between $\mathbf{x}^{(0)}$ and $\mathbf{x}^*$. The error bound ϵ is also a function of a quantity we call L , which is an upper bound on the magnitude $\|(\nabla f)(\mathbf{x})\|$ of the gradient, so that

$$\|(\nabla f)(\mathbf{x})\| \leq L, \quad (33.21)$$

where $\mathbf{x}$ ranges over all the points $\mathbf{x}^{(0)}, \dots, \mathbf{x}^{(T-1)}$ whose gradients are computed by GRADIENT-DESCENT. Of course, we don't know the values of L and R , but for now let's assume that we do. We'll discuss later how to remove these assumptions. The analysis of GRADIENT-DESCENT is summarized in the following theorem.

Theorem 33.8

Let $\mathbf{x}^* \in \mathbb{R}^n$ be the minimizer of a convex function f , and suppose that an execution of $\text{GRADIENT-DESCENT}(f, \mathbf{x}^{(0)}, \gamma, T)$ returns $\mathbf{x}\text{-avg}$, where $\gamma = R/(L\sqrt{T})$ and R and L are defined in equations (33.20) and (33.21). Let $\epsilon = RL/\sqrt{T}$. Then we have $f(\mathbf{x}\text{-avg}) - f(\mathbf{x}^*) \leq \epsilon$. ■

We now prove this theorem. We do not give an absolute bound on how much progress each iteration makes. Instead, we use a potential function, as in Section 16.3. Here, we define a potential $\Phi(t)$ after computing $\mathbf{x}^{(t)}$, such that $\Phi(t) \geq 0$ for $t = 0, \dots, T$. We define the *amortized progress* in the iteration that computes $\mathbf{x}^{(t)}$ as

$$p(t) = f(\mathbf{x}^{(t)}) - f(\mathbf{x}^*) + \Phi(t+1) - \Phi(t). \quad (33.22)$$

Along with including the change in potential ($\Phi(t+1) - \Phi(t)$), equation (33.22) also subtracts the minimum value $f(\mathbf{x}^*)$ because ultimately, you care not about the values $f(\mathbf{x}^{(t)})$ but about how close they are to $f(\mathbf{x}^*)$. Suppose that we can show that $p(t) \leq B$ for some value B and $t = 0, \dots, T-1$. Then we can substitute for $p(t)$ using equation (33.22), giving

$$f(\mathbf{x}^{(t)}) - f(\mathbf{x}^*) \leq B - \Phi(t+1) + \Phi(t). \quad (33.23)$$

Summing inequality (33.23) over $t = 0, \dots, T-1$ yields

$$\sum_{t=0}^{T-1} (f(\mathbf{x}^{(t)}) - f(\mathbf{x}^*)) \leq \sum_{t=0}^{T-1} (B - \Phi(t+1) + \Phi(t)).$$

Observing that we have a telescoping series on the right and regrouping terms, we have that

$$\left(\sum_{t=0}^{T-1} f(\mathbf{x}^{(t)}) \right) - T \cdot f(\mathbf{x}^*) \leq TB - \Phi(T) + \Phi(0).$$

Dividing by T and dropping the positive term $\Phi(T)$ gives

$$\frac{\sum_{t=0}^{T-1} f(\mathbf{x}^{(t)})}{T} - f(\mathbf{x}^*) \leq B + \frac{\Phi(0)}{T}, \quad (33.24)$$

and thus we have

$$\begin{aligned} f(\mathbf{x}\text{-avg}) - f(\mathbf{x}^*) &= f\left(\frac{\sum_{t=0}^{T-1} \mathbf{x}^{(t)}}{T}\right) - f(\mathbf{x}^*) \quad (\text{by the definition of } \mathbf{x}\text{-avg}) \\ &\leq \frac{\sum_{t=0}^{T-1} f(\mathbf{x}^{(t)})}{T} - f(\mathbf{x}^*) \quad (\text{by Lemma 33.7}) \\ &\leq B + \frac{\Phi(0)}{T} \quad (\text{by inequality (33.24)}) . \end{aligned} \quad (33.25)$$